Software Requirements Specification (SRS)

Document control

Field	Value
Project	SentinelPay
Document	Software Requirements Specification
Version	1.0
Prepared	22 October 2026
System type	Django server-rendered educational demonstration
Status	Describes current implemented scope and identified limitations

1. Introduction

1.1 Purpose

This SRS records the current functional and non-functional requirements for SentinelPay. It is intended for developers, reviewers, testers, and project stakeholders who need a common description of the system's capabilities, boundaries, and expected behavior.

1.2 Scope

SentinelPay demonstrates customer sign-in, synthetic account viewing, payment creation and review, deterministic fraud screening, simulated payment outcomes, ledger-backed debits, transaction history, CSV statements, fraud case review, and append-only audit events.

The product is not a production banking system. It does not connect to financial institutions, move real funds, or provide regulated financial services. The default database is SQLite; optional MySQL configuration is present but unverified.

1.3 Definitions

Term	Definition
Customer	Authenticated Django user who can view their own accounts and payments.
Beneficiary	Payment destination record. It may belong to one user or be project-wide (**owner** is empty).
Project-wide beneficiary	Beneficiary available to every authenticated user; it does not make payments or accounts project-wide.
Payment	Persisted request with amount, owner, status, fraud evidence, and simulator outcome.
Ledger entry	Persisted posting representing the debit made after simulator success.
Fraud score	Sum of triggered demonstration rule points, capped at 100; not a probability.
Fraud case	Review record created when a payment is held or rejected by fraud policy.
Idempotency key	Customer-scoped request key used to prevent duplicate payment creation/debit.
Simulator	Deterministic local gateway adapter; it never contacts a real payment network.
Audit event	Append-only record of payment-processing or fraud-case activity.

1.4 References

? **README.md** — setup instructions, fraud policy, outcomes, and limitations.

? **docs/01-project-flow.md** — end-to-end browser and server flow.

? **docs/02-project-structure.md** — source code map and architectural rationale.

? **sentinelpay/urls.py**, **payments/urls.py** — current route definitions.

? **sentinelpay/settings.py** — current runtime and database configuration.

? Django documentation for the installed Django release and built-in authentication/admin components.

2. Overall description

2.1 Product perspective

SentinelPay is a single Django web application. Django handles sessions, forms, templates, ORM persistence, authentication, and administration. The product-specific apps are banking, payments, fraud_engine, and audit. CSS is served from the project static directory. There is no separate SPA frontend, mobile app, public API, external payment gateway, or third-party fraud service.

2.2 User classes

User class	Capabilities
Unauthenticated visitor	View the sign-in page and submit credentials. Cannot access customer or operational data.
Authenticated customer	View own accounts/dashboard, create and review simulated payments, view own payment receipts/history, and export own statement.
Fraud reviewer	Customer capabilities plus fraud monitoring and case review when the user is Django staff/superuser or belongs to **Fraud Analysts**.
Django administrator	Access Django admin when granted Django staff permissions; administer configured records and user/group access.

2.3 Operating environment

? Python and Django versions are pinned in **requirements.txt**.

? Supported project setup is documented for Windows PowerShell.

? SQLite is selected unless **DB_ENGINE=mysql** is explicitly set.

? The application is served using Django's WSGI interface or development server.

? A modern browser with HTML form, cookie, and CSS support is required.

? There is no external browser JavaScript dependency for the documented core workflows.

2.4 Constraints and assumptions

? Customers, accounts, and beneficiary records are synthetic demonstration records.

? The available merchant codes select deterministic local outcomes.

? Django environment settings are provided through process environment variables; **.env.example** is a placeholder and is not automatically loaded.

? Users and fraud-review permissions are provisioned by an administrator or development setup; there is no registration feature.

? Fraud policy limits and weights are educational configuration and are not production controls.

? A deployment must configure a secure secret, allowed hosts, HTTPS, database credentials, and production-appropriate Django settings.

3. System features and functional requirements

Priority definitions: **Must** is required for the current demonstrated behavior; **Should** describes an important usability or operational expectation; **Out of scope** is deliberately not implemented.

3.1 Authentication and session management

? **FR-AUTH-01 (Must):** The system shall authenticate users using Django's username/password authentication.

- **FR-AUTH-02 (Must):** The system shall require authentication for the overview, payment, history, receipt, and statement routes.
- **FR-AUTH-03 (Must):** Successful login shall redirect the authenticated user to the configured landing page.
- **FR-AUTH-04 (Must):** An authenticated user shall be able to sign out using a CSRF-protected POST and shall lose access to authenticated pages until signing in again.
- **FR-AUTH-05 (Must):** Invalid credentials shall not establish an authenticated session and shall return an error on the sign-in page.
- **FR-AUTH-06 (Out of scope):** Public registration, email verification, password reset, MFA, and identity proofing are not provided.

3.2 Customer account overview

- **FR-ACCT-01 (Must):** The overview shall show only bank accounts owned by the authenticated user.
- **FR-ACCT-02 (Must):** The overview shall mask account references in customer-facing account cards and receipts.
- **FR-ACCT-03 (Must):** Payment counts, recent payments, monthly spend, and six-month activity shall be calculated from the current user's stored payment records.
- **FR-ACCT-04 (Must):** Empty account and payment results shall be presented without inventing account balances or transactions.

3.3 Payment creation and review

- **FR-PAY-01 (Must):** A customer shall be able to begin a payment by selecting an owned source account, eligible beneficiary if desired, positive amount, and configured simulator merchant code.
- **FR-PAY-02 (Must):** The system shall limit account and beneficiary choices using server-side queryset validation.
- **FR-PAY-03 (Must):** A customer may use a beneficiary owned by that customer or a project-wide beneficiary. The system shall reject another customer's private beneficiary.
- **FR-PAY-04 (Must):** Valid payment details shall be stored as a customer session draft and shown on a review page before any payment is created.
- **FR-PAY-05 (Must):** The review page shall allow the customer to confirm, cancel, or return to edit details.
- **FR-PAY-06 (Must):** Cancelling before confirmation shall discard the draft and shall not create a payment or ledger posting.
- **FR-PAY-07 (Must):** Review confirmation shall revalidate the session draft and reject missing or invalid saved data.
- **FR-PAY-08 (Must):** The review interface shall distinguish merchant/simulator route from the unique transaction reference, which is generated only when payment processing creates the payment.
- **FR-PAY-09 (Must):** The system shall use decimal monetary fields and reject non-positive payment amounts.
- **FR-PAY-10 (Out of scope):** Payment fees, recipient bank transfer details management, customer-defined limits, and real payment-provider submission are not available.

3.4 Idempotency, payment integrity, and outcomes

- **FR-INT-01 (Must):** The system shall associate an idempotency key with a customer and payment request fingerprint.
- **FR-INT-02 (Must):** Repeating the same key and same payment request shall return the original payment without creating another debit.
- **FR-INT-03 (Must):** Reusing a key for a different payment request shall be rejected as a conflict.
- **FR-INT-04 (Must):** The system shall validate source-account ownership and available balance on the server during confirmation.
- **FR-INT-05 (Must):** Payment statuses shall transition only through the defined state machine.
- **FR-INT-06 (Must):** A successful simulator response shall post one negative ledger entry and decrement the source-account balance once.
- **FR-INT-07 (Must):** Declines, errors, fraud holds/rejections, and timeouts shall not debit the source account.

? **FR-INT-08 (Must):** A timeout shall be represented as unknown and shall not be retried automatically.

? **FR-INT-09 (Out of scope):** Gateway reconciliation for unknown payments and double-entry accounting are not implemented.

3.5 Fraud screening

? **FR-FRAUD-01 (Must):** The payment service shall construct fraud context from server-side payment history, amount, beneficiary creation time, and local payment time.

? **FR-FRAUD-02 (Must):** The fraud engine shall evaluate the configured deterministic rules and persist score, decision, ruleset version, and triggered rule explanations.

? **FR-FRAUD-03 (Must):** The server shall apply the fraud decision before the simulator is called.

? **FR-FRAUD-04 (Must):** A decline decision shall reject the payment; review/step-up decisions shall hold it without a debit.

? **FR-FRAUD-05 (Must):** A held or rejected payment shall create a fraud case and corresponding audit event.

? **FR-FRAUD-06 (Must):** Receipt, monitoring, and case pages shall display recorded backend risk data, not a client-generated score.

? **FR-FRAUD-07 (Must):** The system shall identify inactive device and IP-country signals as unavailable; it shall not fabricate those signals.

? **FR-FRAUD-08 (Out of scope):** Machine-learning scoring, external fraud feeds, step-up challenge, device profiling, and IP geolocation are not implemented.

3.6 Transaction history, receipt, and statements

? **FR-HIST-01 (Must):** A customer shall see only their own payment history and receipt records.

? **FR-HIST-02 (Must):** Transaction filters shall be validated and applied on the server.

? **FR-HIST-03 (Must):** A receipt shall show the persisted status, reference, amount, account, recipient/merchant, fraud result, processing events, and ledger activity where present.

? **FR-HIST-04 (Must):** The transaction reference shall be unique and shall be used to find the receipt.

? **FR-HIST-05 (Must):** A CSV statement shall be restricted to the authenticated customer's payments and derive debit totals from ledger entries.

? **FR-HIST-06 (Must):** Missing payment or ledger data shall be presented as unavailable/empty rather than as a fabricated success.

3.7 Fraud case operations

? **FR-CASE-01 (Must):** Fraud-center and fraud-case views shall require server-side reviewer permission.

? **FR-CASE-02 (Must):** The fraud center shall aggregate persisted payment and case data and support validated server-side filters.

? **FR-CASE-03 (Must):** A reviewer shall be able to update a case only through an allowed status transition.

? **FR-CASE-04 (Must):** A successful case update shall assign the reviewer and append an audit record with old/new status and reviewer note.

? **FR-CASE-05 (Must):** Unauthorized customers shall receive HTTP 403 from protected fraud operations even if they navigate directly to a URL.

3.8 Audit trail

? **FR-AUD-01 (Must):** Payment processing stages, fraud evaluation/case creation, and case updates shall append audit events.

? **FR-AUD-02 (Must):** Payment events shall carry a correlation reference so they can be displayed with the associated payment.

? **FR-AUD-03 (Must):** Audit rows shall be append-only through the application model interface.

? **FR-AUD-04 (Out of scope):** A standalone audit browser/export route is not currently provided.

4. External interface requirements

4.1 User interface

- **UI-01:** The application shall use shared templates and coherent SentinelPay navigation.
- **UI-02:** Pages shall adapt to desktop and mobile viewports.
- **UI-03:** Forms shall include visible labels, validation feedback, and CSRF protection.
- **UI-04:** Risk meters, timelines, and charts shall reflect data supplied by backend views.
- **UI-05:** CSS animations shall not be required to understand content or transaction status.
- **UI-06:** The stylesheet shall honor **prefers-reduced-motion**.
- **UI-07:** Sandbox/synthetic context shall remain clear where it affects user interpretation.

4.2 Software interfaces

- **SI-01:** Django ORM shall persist users, accounts, beneficiaries, payments, cases, ledger entries, and audit events.
- **SI-02:** The simulator interface shall be local and deterministic. No network payment integration is configured.
- **SI-03:** Django's built-in authentication, session, message, and admin components shall be used where configured.

4.3 Communication and security interfaces

- **CI-01:** Browser form submissions that change state shall use Django CSRF protection.
- **CI-02:** Authenticated pages shall use Django's session cookie and authentication middleware.
- **CI-03:** Production deployments shall provide valid secret key, allowed hosts, HTTPS, and secure database credentials through environment configuration.

5. Non-functional requirements

5.1 Security and privacy

- **NFR-SEC-01:** Access control shall be enforced server-side for authenticated customer data and fraud-review routes.
- **NFR-SEC-02:** Querysets for customer payment, account, statement, and receipt data shall be scoped to the logged-in user.
- **NFR-SEC-03:** The server—not client-submitted values—shall determine account/beneficiary eligibility, balance, fraud result, status, and debit.
- **NFR-SEC-04:** Secrets shall not be committed. **.env.example** shall contain placeholders, not credentials.
- **NFR-SEC-05:** Local database contents, environment files, and credentials shall not be distributed in the source ZIP.
- **NFR-SEC-06:** Production deployment shall replace development defaults and enable secure cookie/transport settings.

5.2 Data integrity

- **NFR-DATA-01:** Monetary amounts shall use fixed-precision decimal fields.
- **NFR-DATA-02:** Database constraints shall protect positive payment amounts, non-negative balances, per-customer idempotency uniqueness, and unique ledger posting per payment/account.
- **NFR-DATA-03:** Payment creation and debit processing shall run through transactional server-side service code.
- **NFR-DATA-04:** Schema changes shall be represented as Django migrations and reviewed before applying to an existing data store.

5.3 Usability and accessibility

- **NFR-UX-01:** Shared navigation and page styling shall provide consistent wayfinding.
- **NFR-UX-02:** Keyboard focus shall be visible on interactive controls.
- **NFR-UX-03:** Layouts shall support small screens without horizontal page overflow for standard pages.
- **NFR-UX-04:** Reduced-motion preferences shall disable decorative CSS animation.

? **NFR-UX-05:** Empty and error states shall not imply success where no successful backend result exists.

5.4 Maintainability and testability

? **NFR-MAINT-01:** HTTP views, forms, models, payment orchestration, fraud evaluation, and audit services shall remain separated by responsibility.

? **NFR-MAINT-02:** Fraud rules and state transitions shall be testable independently of browser presentation.

? **NFR-MAINT-03:** Automated tests shall cover payment outcomes, risk behavior, ownership, idempotency, reviewer permissions, audit recording, and data scope.

? **NFR-MAINT-04:** External provider integration shall be introduced behind an explicit adapter rather than mixed into templates or fraud rules.

5.5 Performance and availability

? **NFR-PERF-01:** Dashboard and monitoring pages shall calculate from database queries and should remain responsive at demonstration scale.

? **NFR-PERF-02:** Fraud monitoring currently limits displayed payment and case querysets to bounded recent records; this is not a large-scale analytics service.

? **NFR-AVAIL-01:** Availability, throughput, recovery time, and production service-level guarantees are not defined for this academic demonstration.

6. Data requirements

Entity	Key relationships and data
User	Django authentication identity; owns accounts, private beneficiaries, and payments; may have reviewer group/staff permissions.
BankAccount	Owned by one user; synthetic account reference, currency, balance, creation time.
Beneficiary	May be owned by one user or have no owner for project-wide scope; name, synthetic destination reference, creation time.
Payment	Belongs to customer and source account; may link a beneficiary; stores amount/currency, route, idempotency data, status, fraud outcome, gateway outcome, timestamps.
LedgerEntry	Links one account and payment to a nonzero signed amount; payment debit is negative.
FraudCase	One-to-one with a held or rejected payment; stores status, severity, reviewer, and timestamps.
AuditEvent	Optional actor; action, object/correlation references, metadata, timestamp; append-only.

Database schema is evolved through the migrations in the respective app directories. The source distribution omits local SQLite data to avoid exposing machine-specific records.

7. Key workflows and state behavior

7.1 Payment states

Current state	Allowed next state(s)
INITIATED	**REJECTED**, **HELD_FOR_REVIEW**, **SUBMITTED**
SUBMITTED	**SUCCEEDED**, **FAILED**, **UNKNOWN**
HELD_FOR_REVIEW	Terminal in the payment state machine
REJECTED	Terminal
SUCCEEDED	Terminal
FAILED	Terminal
UNKNOWN	Terminal until a reconciliation feature is implemented

7.2 High-level sequence

```
Visitor -> Login -> Authenticated customer -> Overview
-> Enter payment -> Server form validation -> Session draft
-> Review/edit/cancel -> Confirm
-> Ownership + balance + idempotency validation
-> Fraud rules evaluate -> reject / hold / submit to simulator
-> Payment status + audit events (+ ledger debit only on success)
-> Customer receipt
-> Sign out -> Session ends
```

8. Routes and permissions

Route	Function	Access
/login/	Sign in	Public
/logout/	Sign out	Authenticated session; POST
/	Customer overview	Authenticated
/payments/new/	Enter payment details	Authenticated
/payments/review/	Review/confirm session draft	Authenticated
/payments/	Payment history	Authenticated; own data only
/payments/statement.csv	CSV statement	Authenticated; own data only
/payments/<uuid>/	Payment receipt	Authenticated; own payment only
/payments/fraud/	Monitoring and review queue	Staff/superuser/Fraud Analysts
/payments/fraud/cases/<id>/	Case details and update	Staff/superuser/Fraud Analysts
/admin/	Django administration	Django staff permissions

9. Verification and acceptance criteria

The implementation is acceptable for the current scope when:

1. Anonymous access to protected customer routes redirects to login.
2. A customer cannot see another user's account, payment, receipt, or statement data.
3. A customer can select their own or project-wide beneficiary but not another user's private beneficiary.
4. Payment review happens before payment creation, and cancellation creates no payment.
5. Repeating a confirmed request with the same key does not debit twice; reusing the key for changed data is rejected.
6. Fraud decisions are generated on the server, persist on the payment, and control whether the gateway is called.
7. Only simulator success creates one ledger debit; timeout/failure/hold/rejection do not.
8. Fraud operations reject unauthorized users and append audit records for permitted case changes.
9. Dashboard and fraud visualizations display stored backend data.
10. **python manage.py check** and the automated pytest suite pass.

The test suite currently has coverage for fraud-engine rules and payment behavior. Browser rendering has been inspected for the sign-in page at mobile and desktop sizes; that visual check does not substitute for automated browser coverage of every route.

10. Out-of-scope features and known limitations

- ? Real banking/provider transfers, real balances, and external payment APIs.
- ? Account opening, customer registration, self-service password reset, MFA, and customer settings.
- ? Payment fees, configured per-transaction limits, and step-up authentication.
- ? Device fingerprinting, IP geolocation, external fraud intelligence, or ML scoring.
- ? Gateway reconciliation for unknown outcomes and automatic retry.
- ? Customer-managed beneficiary administration.
- ? Standalone audit-history interface and real-time notifications.
- ? Production MySQL verification, high availability, formal SLA, and load/performance targets.

11. Change management

Changes to routes, data model, fraud thresholds, payment outcomes, or user roles should update this SRS, the project flow/structure documentation where affected, Django migrations when schema changes are involved, and relevant automated tests. Existing SQLite data should be backed up and migrations reviewed before production-like schema changes.